

Dockerization Summary & Mental Map

The Mental Map of Dockerizing

Think of your project like a Restaurant. Before Docker, you were running everything in your home kitchen (your laptop). Docker lets you package this restaurant so it can be deployed anywhere identically.

1. The Recipe Card (Dockerfile)

The Dockerfile is the step-by-step recipe to build the kitchen itself.

- Base Image (FROM python:3.13-slim): We start with a clean, empty kitchen that has Python installed.
- Ingredients (COPY requirements.txt ...): We buy all the groceries (libraries like Django) and put them in the cupboards.
- Cookware (RUN apt-get install ...): We install necessary tools (like gcc and mysql clients) to cook the food.
- The Code (COPY . /app): We bring in your secret family recipes (your source code).
- The Chef (ENTRYPOINT/CMD): We tell the kitchen what to do when it opens (run migrations, then start the server).

2. The Restaurant Manager (docker-compose.yml)

A real restaurant needs more than just a kitchen; it needs a pantry, a freezer, and a cleaning crew. docker-compose manages this team.

- Web: Your Django app (the Kitchen).
- DB (MySQL): The Pantry where data lives.
- Redis: The fast-access Counter/Freezer for catching and quick tasks.
- Worker: The Cleaning Crew/Prep Cooks (Celery) doing background work.
- Beat: The Manager who shouts orders at specific times (Scheduled tasks).
- Networks: The hallways allowing these rooms to talk to each other.

3. The Configuration Bridge (prod.py & .env)

Your code needs to know it's not at home anymore.

- Environment Variables: Instead of hardcoding 'localhost', we use variables like DB_HOST=db. This tells Django: 'The database is in the room named db'.
- Updates: The 'prod.py' file was modified to allow Django to read these signs posted on the wall by Docker.

Summary of Changes

- Fixed Missing Dependency: Added 'djoser' to pyproject.toml so the authentication system works.
- Created Dockerfile: Defines the 'Storefront API' machine.
- Created docker-compose.yml: Spins up the App, MySQL, Redis, and Celery together.
- Updated settings/prod.py: Made it flexible to accept configuration from Docker.

You can now build and run your project using: `docker-compose up --build`